Booking Lite
Proje Dokümanı

Spring Boot (Java) + React.js | PostgreSQL | JWT | Redis

İçindekiler

1. Proje Tanımı
2. Amaç ve Hedefler
3. Teknik Kapsam ve Mimari
4. Kullanıcı Akışı (Ekranlar ve İşleyiş)
5. API Kapsamı ve İş Kuralları
6. Veri Modeli (Tablolar ve Alanlar)
7. DTO Ayrımı ve Mapping Kuralları
8. Validasyon Kuralları
9. Güvenlik Gereksinimleri
10. Cache, Rate Limiting ve Performans
11. Loglama ve Gözlemlenebilirlik (Opsiyonel)
12. Test, Kalite ve Teslimat

1. Proje Tanımı

Booking Lite; müşterilerin şehir ve tarih aralığına göre otel arayıp uygun oda tiplerini görüntüleyebildiği ve rezervasyon oluşturabildiği, otel sahiplerinin (partner) kendi otellerini, oda tiplerini, fiyat ve stok takvimlerini yönetebildiği web tabanlı bir rezervasyon uygulamasıdır.

Uygulama üç ana yüzeyden oluşur: Müşteri arayüzü, Partner paneli ve Admin paneli. Kimlik doğrulama JWT ile yapılır. Veriler PostgreSQL üzerinde tutulur. Redis, cache ve rate limiting gibi yüksek trafikli alanlarda kullanılır.

Medya (foto/video) yönetimi dosyaları veritabanında saklamak yerine object storage yaklaşımı ile ele alınır. Veritabanında yalnızca dosya URL'si ve metadata tutulur.

2. Amaç ve Hedefler

Bu projenin hedefleri:

- Gerçek dünyaya yakın arama ve rezervasyon akışını uçtan uca geliştirmek.
- Rol bazlı yetkilendirme ile müşteri, partner ve admin senaryolarını doğru ayırmak.
- Gün bazlı fiyat ve stok yönetimi (rate calendar) ile otel tarafındaki operasyonu desteklemek.
- Eşzamanlı rezervasyonlarda double-booking riskini transaction ve kilitleme ile engellemek.
- Cache, rate limiting ve güvenlik kontrollerini uygulayarak üretim ortamına uygun bir temel kurmak.

3. Teknik Kapsam ve Mimari

3.1 Teknolojiler

- Backend: Java 21, Spring Boot 3.x, Spring Security, Spring Data JPA/Hibernate
- Veritabanı: PostgreSQL, migration için Flyway (veya Liquibase)
- Cache ve rate limiting: Redis (Lettuce; gerekirse distributed lock için Redisson)
- Frontend: React.js, React Router; UI kütüphanesi olarak MUI veya seçilen bir tema
- Dokümantasyon: OpenAPI (Swagger)
- Opsiyonel: Docker Compose, MinIO (medya), OpenTelemetry Collector, Seq

3.2 React arayüz teması (opsiyonel)

Arayüzün hızlı ve tutarlı ilerlemesi için hazır bir hotel teması kullanılabilir. Tema seçimi zorunlu değildir; isterse temel bir tasarım ile de ilerlenebilir. Hazır tema kullanılacaksa, seçilen HTML teması React bileşenlerine bölünerek yeniden kullanılabilir hale getirilir.

Örnek kaynak: https://themewagon.com/theme-tag/hotel-template/

3.3 Modüler yapı (öneri)

- auth (JWT, RBAC, kullanıcı yönetimi)

- hotels (otel bilgileri ve yayın durumu)
- media (foto/video metadata, upload akışı)
- rooms (oda tipleri, kapasite, özellikler)
- rates (fiyat ve stok takvimi)
- search (otel arama ve filtreleme)
- reservations (rezervasyon oluşturma, iptal)
- admin (partner doğrulama, otel moderasyonu)
- infra (redis, storage, ortak yardımcılar)

3.4 Yetki matrisi (özet)

MVP için minimum erişim kuralları:

- CUSTOMER: Arama yapabilir, otel detaylarını görebilir, rezervasyon oluşturabilir, kendi rezervasyonlarını listeleyip iptal edebilir.
- PARTNER: Kendi otelini/medyasını/oda tiplerini yönetebilir, rate calendar güncelleyebilir, kendi oteline ait rezervasyonları görebilir.
- ADMIN: Partner doğrulayabilir, otelleri yayınlayabilir/askıya alabilir, sistem genelinde raporlama ve denetim kayıtlarını görebilir.

4. Kullanıcı Akışı (Ekranlar ve İşleyiş)

4.1 Giriş Ekranı

Kullanıcı e-posta ve şifre ile giriş yapar. Başarılı girişten sonra rolüne göre ilgili panele yönlendirilir. Hatalı girişte kullanıcıya anlaşılır hata mesajı gösterilir.

4.2 Kayıt Olma Ekranı

Kayıt ekranında e-posta ve şifre ile yeni hesap oluşturulur. Varsayılan rol CUSTOMER olarak atanır. Partner rolü, ayrı bir başvuru akışı ile veya admin onayı ile verilir.

4.3 Rol Bazlı Yönlendirme

- CUSTOMER: Arama ekranına yönlendirilir.
- PARTNER: Partner Dashboard ekranına yönlendirilir.
- ADMIN: Admin Dashboard ekranına yönlendirilir.

4.4 Müşteri Akışı

4.4.1 Arama Ekranı

Alanlar: Şehir, giriş tarihi, çıkış tarihi, yetişkin sayısı, çocuk sayısı (opsiyonel).

Kurallar: Çıkış tarihi giriş tarihinden sonra olmalıdır.

4.4.2 Sonuç Listesi
Sonuç listesinde otel adı, kapak görseli, seçilen tarih aralığına göre başlangıç fiyatı ve uygunluk bilgisi gösterilir. Filtreler: fiyat aralığı, kapasite, (opsiyonel) yıldız/puan.

4.4.3 Otel Detay + Oda Seçimi
Otel detay sayfasında galeri (foto/video), açıklama, konum bilgisi ve oda tipleri listelenir. Seçilen tarihler için her oda tipinin toplam fiyatı ve kalan stok bilgisi gösterilir.

4.4.4 Rezervasyon Oluşturma
Müşteri bir oda tipi seçerek rezervasyonu onaylar. Rezervasyon oluşturma sırasında backend tarafında stok düşümü transaction içinde yapılır ve double-booking engellenir.

4.4.5 Rezervasyonlarım
Müşteri kendi rezervasyonlarını listeler, detaylarını görür ve politika uygunsa iptal eder.

4.5 Partner Akışı (Partner Paneli)

4.5.1 Partner Dashboard
Özet kartlar: yaklaşan rezervasyonlar, bugünkü girişler, iptaller (opsiyonel).

4.5.2 Otel Yönetimi
Partner kendi otel bilgilerini yönetir: ad, şehir, adres, açıklama, check-in/out saatleri, politikalar. Otel yayına alınmadan önce admin onayı gerektirebilir.

4.5.3 Medya Yönetimi (Foto/Video)
Fotoğraf ve video ekleme, kapak görseli seçme, sıralama ve silme işlemleri yapılır. Dosyalar object storage üzerinde tutulur, veritabanında sadece URL ve metadata saklanır.

4.5.4 Oda Tipi Yönetimi
Oda tipi ekleme/düzenleme/silme: kapasite, yatak bilgisi, özellikler, açıklama.

4.5.5 Fiyat ve Stok Takvimi (Rate Calendar)
Partner oda tipine göre gün bazlı fiyat ve stok yönetir. Bazı günlerde satış kapatma (stop sell) ve minimum konaklama (min stay) gibi kurallar opsiyoneldir.

4.5.6 Rezervasyon Yönetimi
Partner kendi oteline ait rezervasyonları listeler. Gerekirse misafir notlarını görüntüler ve iptal taleplerini yönetir.

4.6 Admin Akışı (Admin Paneli)

4.6.1 Admin Dashboard
Özet: toplam kullanıcı, toplam otel, yayındaki otel sayısı, bugünkü rezervasyon sayısı.

4.6.2 Partner Onay

Partner başvuruları incelenir ve uygun olanlar onaylanır. Onaylanan kullanıcı PARTNER rolü ile aktif edilir.

4.6.3 Otel Moderasyonu

Otel yayın durumu yönetilir: DRAFT, PUBLISHED, SUSPENDED. Uygunsuz içerikler askıya alınabilir.

5. API Kapsamı ve İş Kuralları

API tasarımında tutarlı isimlendirme, sayfalama, standart hata modeli ve rol bazlı erişim kontrolü temel alınır. Tüm endpoint'ler /api/v1 altında versiyonlanabilir.

5.1 Temel API modülleri

- Auth: register, login, refresh (opsiyonel), logout
- Hotel: otel CRUD (partner), otel yayın durumu (admin)
- Media: upload akışı, listeleme, sıralama, silme
- RoomType: oda tipi CRUD (partner)
- RateCalendar: gün bazlı fiyat ve stok güncelleme (partner)
- Search: şehir+tarih+kapasite ile arama, filtreleme, sayfalama
- Reservation: rezervasyon oluşturma, listeleme, iptal

5.2 Rezervasyon iş kuralı (double-booking önleme)

Rezervasyon oluşturma sırasında check-in ile check-out arasındaki her gece için stok düşümü yapılır. Bu işlem tek transaction içinde gerçekleşir. İlgili rate calendar satırları kilitlenir ve her gün için available_units kontrol edilir. Herhangi bir gün stok yetersizse rezervasyon reddedilir ve stok düşümü yapılmaz.

5.3 Medya yükleme yaklaşımı

Medya yükleme için iki yöntemden biri seçilebilir: Sunucu üzerinden upload veya object storage'a imzalı URL ile doğrudan upload. İmzalı URL yaklaşımı büyük dosyalarda daha verimlidir ve backend yükünü azaltır. Her iki durumda da dosya türü ve boyutu sınırlandırılır.

5.4 Opsiyonel ürün özellikleri

MVP sonrasında eklenebilecek özellikler:

- Otel puanlama ve yorumlar (moderasyonlu).
- Favorilere ekleme ve son aramalar.
- Kupon/indirim (promotions) ve dinamik fiyat kuralları.
- Çoklu dil desteği ve para birimi dönüştürme.
- E-posta bildirimleri (rezervasyon onayı/iptali).

5.5 Hata modeli

API hata cevapları standart bir formatta dönmelidir. Validasyon hataları alan bazlı listelenmeli, yetkisiz erişimler 401/403 ile ayrıştırılmalıdır. Sunucu hatalarında hassas bilgi sızdırılmamalıdır.

6. Veri Modeli (Tablolar ve Alanlar)

Aşağıdaki tablolar minimum kapsam için yeterlidir. Gerekli görülen alanlar zaman içinde genişletilebilir, ancak ilk etapta karmaşıklığı kontrollü tutmak esastır.

6.1 users

- id
- email
- password_hash
- role
- status
- created_at

6.2 partner_profiles

- id
- user_id
- company_name
- verified
- created_at

6.3 hotels

- id
- partner_user_id
- name
- city
- address
- description
- checkin_time
- checkout_time
- status
- created_at

6.4 hotel_media

- id
- hotel_id
- type (PHOTO/VIDEO)
- url

- sort_order
- is_cover
- created_at

6.5 room_types

- id
- hotel_id
- name
- capacity_adults
- capacity_children
- amenities_json
- created_at

6.6 rate_calendar

- id
- room_type_id
- date
- available_units
- price
- currency
- min_stay (ops.)
- stop_sell (ops.)
- updated_at
- unique(room_type_id, date)

6.7 reservations

- id
- hotel_id
- room_type_id
- user_id
- checkin
- checkout
- guests_adults
- guests_children
- total_price
- status
- created_at

6.8 reservation_nights

- id
- reservation_id

- date
- price

6.9 audit_log

- id
- actor_user_id
- entity
- entity_id
- action
- payload_json
- created_at

7. DTO Ayrımı ve Mapping Kuralları

Entity (veritabanı modeli) ile dışarı açılan API modelleri birbirinden ayrılmalıdır. Request ve response DTO'ları entity sınıflarını doğrudan dışarı taşımamalıdır.

Uygulanacak prensipler:

- Her endpoint için ihtiyaç kadar alan içeren DTO tanımlanır; gereksiz alanlar taşınmaz.
- Response DTO'larında iç sistem alanları (ör. password hash) kesinlikle yer almaz.
- Mapping tek bir yerde toplanır: manuel mapper sınıfları veya MapStruct gibi bir araç kullanılabilir.
- Create/Update işlemlerinde entity üzerinde izin verilen alanlar dışında alan set edilmez (mass assignment riskine dikkat).
- Liste DTO'ları ile detay DTO'ları ayrılır; performans ve veri sızıntısı açısından kontrol sağlanır.

8. Validasyon Kuralları

8.1 Kullanıcı ve kimlik doğrulama

- Email formatı doğrulanmalı ve unique olmalıdır.
- Şifre en az 8 karakter olmalı; basit kurallar (sayı/harf) uygulanabilir.

8.2 Otel

- Otel adı zorunlu, maksimum 120 karakter.
- Şehir zorunlu.
- Adres zorunlu, maksimum 250 karakter.
- Açıklama maksimum 2000 karakter.

8.3 Oda tipi

- Oda tipi adı zorunlu, maksimum 80 karakter.

- capacity_adults minimum 1 olmalı.
- available_units negatif olamaz.

8.4 Rate calendar

- Fiyat sıfırdan büyük olmalı.
- Stok 0 veya daha büyük olmalı.
- Stop-sell aktif ise ilgili tarihlerde rezervasyon yapılamaz.
- Min-stay tanımlıysa, seçilen tarih aralığı bu değerden kısa olamaz (opsiyonel).

8.5 Rezervasyon

- checkOut tarihi checkIn tarihinden sonra olmalı.
- Her gece için stok yeterli olmalı; yetersizse rezervasyon reddedilmeli.
- Toplam fiyat, rezervasyon anındaki gece fiyatlarına göre hesaplanıp reservation_nights tablosuna yazılmalı.

9. Güvenlik Gereksinimleri

Bu proje üretim ortamına yakın güvenlik beklentileri ile geliştirilmelidir. Minimum gereksinimler:

- JWT ile kimlik doğrulama, rol bazlı yetkilendirme (RBAC) ve endpoint bazlı erişim kontrolleri.
- Parola saklama: BCrypt gibi güçlü hash algoritması; düz metin parola kesinlikle tutulmaz.
- Rate limiting: login, search ve medya upload gibi kritik endpoint'lerde uygulanır.
- Girdi doğrulama: sunucu tarafı validasyon zorunlu; client tarafı sadece kullanıcı deneyimi içindir.
- XSS: kullanıcıdan gelen metin alanları güvenli şekilde render edilir; gerektiğinde sanitize edilir.
- CSRF: JWT header ile kullanılıyorsa risk düşüktür; cookie tabanlı auth tercih edilirse CSRF koruması zorunludur.
- CORS: sadece izin verilen origin'lere açık olacak şekilde yapılandırılır.
- Dosya yükleme güvenliği: izin verilen tipler ve boyut sınırı; metadata kontrolü; gerekirse tarama adımı.
- Hassas veri sızıntısı: hata mesajlarında stack trace veya gizli bilgiler dışarı verilmez.
- Audit log: kritik değişikliklerde kim-ne-zaman bilgisinin kaydı tutulur.

10. Cache, Rate Limiting ve Performans

10.1 Redis cache stratejisi

- Arama sonuçları kısa TTL ile cache'lenir (ör. 30-120 saniye).
- Otel detay verileri daha uzun TTL ile cache'lenir (ör. 5-30 dakika).
- Availability snapshot çok kısa TTL ile cache'lenebilir (ör. 15-60 saniye).
- Cache invalidation: partner otel/oda/fiyat güncellediğinde ilgili cache anahtarları temizlenir.

10.2 Performans notları

- Arama endpoint'leri sayfalama (pagination) desteklemelidir.
- Sık sorgulanan alanlarda indeksler kullanılmalıdır (hotel.city, rate_calendar.room_type_id+date, reservations.hotel_id).
- N+1 sorgu riskine dikkat edilmeli; gerekli yerlerde join fetch veya projection tercih edilmelidir.

10.3 Rate limiting kuralları

- Login: IP başına dakika başına sınırlı deneme.
- Search: IP veya kullanıcı bazında dakika başına istek sınırı.
- Media upload: kullanıcı başına dakika başına upload sınırı ve dosya boyutu limiti.

11. Loglama ve Gözlemlenebilirlik (Opsiyonel)

OpenTelemetry ile trace ve metrik toplama uygulanabilir. Loglar yapılandırılmış formatta tutulur ve Seq üzerinde sorgulanabilir hale getirilir. Her log kaydında traceId ve ilgili iş varlığı kimlikleri (hotelId, roomTypeId, reservationId) bulunmalıdır.

Örnek Seq panelleri:

- Arama endpoint p95 gecikme (latency) ve hata oranı.
- Rezervasyon başarı/başarısızlık oranı; başarısızlığın ana sebepleri (stok yetersizliği, validasyon, yetki).
- Medya upload hata oranı ve boyut dağılımı.

12. Test, Kalite ve Teslimat

Minimum beklentiler:

- Migration dosyaları düzenli ve tekrar çalıştırılabilir olmalıdır.
- En azından rezervasyon transaction akışı için integration test yazılmalıdır.
- Kritik servisler için unit test bulunmalıdır.
- Kod standardı: okunabilir isimlendirme, katmanlar arası net sorumluluk, tekrar eden kodların azaltılması.
- Docker Compose ile yerel geliştirme ortamı tek komutla ayağa kalkabilmelidir (opsiyonel ama tavsiye edilir).